

Komunikacja REST, diagramy sekwencji oraz schematy wiadomości

FlatShare - Checkpoint 3

Grupa projektowa:

Jakub Rak

Bartosz Radomski

Sofia Kuzmenko

Piotr Wójcik

Spis treści

1	Założenia wspólne	3
2	Wspólny mechanizm autoryzacji (JWT)	3
2.1	Zakres stosowania	3
2.2	Wymagane nagłówki HTTP	4
2.3	Proces walidacji tokenu	4
2.4	Obsługa błędów autoryzacji	4
2.5	Uwagi projektowe	4
2.6	Diagram sekwencji – walidacja tokenu JWT	5
3	Logowanie	5
3.1	Opis endpointu	5
3.2	Przykładowe żądanie	6
3.3	Przykładowa odpowiedź — sukces	6
3.4	Obsługa błędów	6
3.5	Diagram sekwencji procesu logowania	7
3.6	Uwagi projektowe	7
4	Wystawienie ogłoszenia	7
4.1	Opis endpointu	7
4.2	Przykładowe żądanie	8
4.3	Przykładowa odpowiedź — sukces	8
4.4	Obsługa błędów	9
4.5	Diagram sekwencji wystawienia ogłoszenia	9
4.6	Uwagi projektowe	9
5	Zarządzanie preferencjami i dopasowywanie	9
5.1	Opis endpointu	10
5.2	Parametry zapytania	10
5.3	Przykładowe żądanie	10
5.4	Przykładowa odpowiedź — sukces	11
5.5	Obsługa sytuacji brzegowych	11
5.6	Diagram sekwencji dopasowywania	12
5.7	Uwagi projektowe	12
6	Wynajem pokoju	12
6.1	Opis endpointu	12
6.2	Przykładowe żądanie	13
6.3	Przykładowa odpowiedź — sukces	13

6.4	Obsługa błędów	14
6.5	Diagram sekwencji wynajmu pokoju	14
6.6	Uwagi projektowe	14

1 Założenia wspólne

Niniejszy dokument opisuje projekt interfejsu programistycznego (API) systemu, ze szczególnym uwzględnieniem komunikacji REST, mechanizmów autoryzacji, diagramów sekwencji oraz schematów wymiany wiadomości.

Na potrzeby projektu przyjęto następujące założenia ogólne:

- API jest dostępne w wersji `/api/v1`.
- Komunikacja pomiędzy klientem a serwerem odbywa się wyłącznie z użyciem protokołu HTTPS.
- Identyfikatory zasobów są reprezentowane jako UUID w postaci tekstowej, zgodnie ze standardem RFC 4122.
- Daty są zapisywane w formacie ISO 8601 (YYYY-MM-DD).
- Daty wraz z czasem są zapisywane w formacie ISO 8601 w strefie UTC (YYYY-MM-DDTHH:MM:SSZ).
- Kwoty pieniężne są reprezentowane jako wartości liczbowe typu dziesiętnego (*decimal*) wraz z kodem waluty zgodnym ze standardem ISO 4217.

Wszystkie endpointy chronione wymagają poprawnej autoryzacji użytkownika z wykorzystaniem mechanizmu JWT. Szczegóły dotyczące procesu walidacji tokenów oraz przekazywania kontekstu użytkownika zostały opisane w osobnym rozdziale, aby uniknąć powielania tej logiki w dalszych scenariuszach biznesowych.

W kolejnych rozdziałach dokumentu przedstawiono zarówno przykładowe żądania i odpowiedzi HTTP, jak i diagramy sekwencji ilustrujące przebieg logiki biznesowej po stronie systemu.

2 Wspólny mechanizm autoryzacji (JWT)

W systemie zastosowano mechanizm autoryzacji oparty o tokeny JWT (*JSON Web Token*), który stanowi wspólny element dla wszystkich endpointów wymagających uwierzytelnienia użytkownika. Mechanizm ten został wydzielony do osobnego scenariusza w celu uniknięcia powielania tej samej logiki w dalszych diagramach sekwencji oraz opisach przypadków użycia.

2.1 Zakres stosowania

Autoryzacja z wykorzystaniem JWT dotyczy wszystkich endpointów chronionych interfejsu API. Każde żądanie do takiego endpointu musi zawierać poprawny token JWT przekazany w nagłówku `HTTP Authorization`.

2.2 Wymagane nagłówki HTTP

Dla endpointów chronionych wymagane są następujące nagłówki:

- **Authorization:** Bearer <JWT>
- **Accept:** application/json

Dodatkowo, opcjonalnie może zostać przekazany nagłówek:

- **X-Request-Id:** <UUID> – identyfikator żądania wykorzystywany w celach diagnostycznych oraz do śledzenia przepływu żądań w systemie.

2.3 Proces walidacji tokenu

Proces weryfikacji tokenu JWT realizowany jest na poziomie filtra autoryzacyjnego, jeszcze przed przekazaniem żądania do warstwy kontrolerów. Logika biznesowa poszczególnych endpointów zakłada, że kontekst użytkownika został już poprawnie ustawiony.

W ramach walidacji tokenu wykonywane są następujące kroki:

1. Przechwycenie przychodzącego żądania HTTP.
2. Odczyt nagłówka **Authorization** oraz wyodrębnienie tokenu JWT.
3. Weryfikacja poprawności podpisu kryptograficznego tokenu.
4. Sprawdzenie daty ważności tokenu.
5. Odczyt identyfikatora użytkownika oraz ról zapisanych w tokenie.
6. Ustawienie kontekstu bezpieczeństwa aplikacji (np. **SecurityContext**).

2.4 Obsługa błędów autoryzacji

W przypadku braku tokenu JWT, przekazania tokenu niepoprawnego lub tokenu, którego ważność wygasła, żądanie zostaje odrzucone, a system zwraca odpowiedź:

- 401 Unauthorized

Żądanie nie jest wówczas przekazywane do dalszych warstw aplikacji.

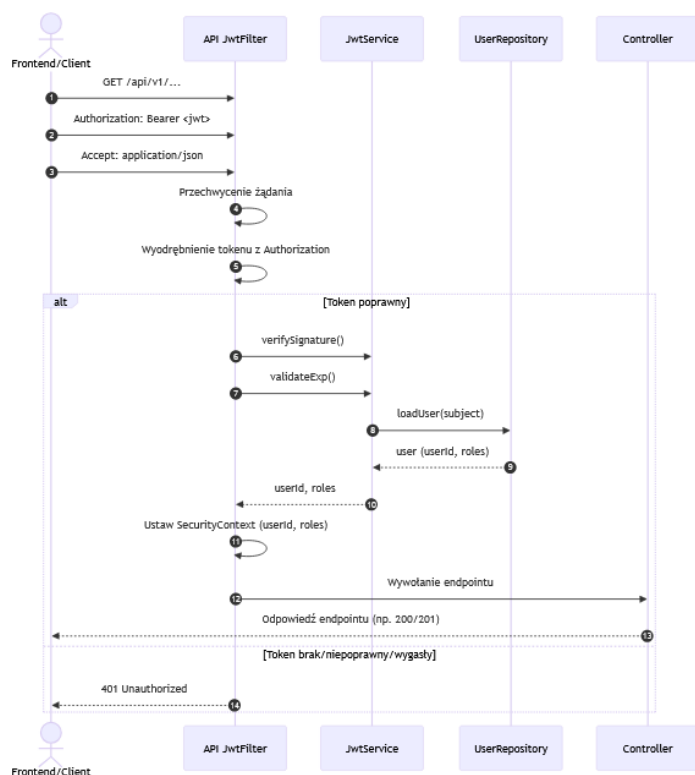
2.5 Uwagi projektowe

Wydzielenie wspólnego mechanizmu autoryzacji pozwala na:

- zwiększenie czytelności dokumentacji,
- uproszczenie diagramów sekwencji opisujących logikę biznesową,
- centralne zarządzanie polityką bezpieczeństwa systemu.

2.6 Diagram sekwencji – walidacja tokenu JWT

Na rysunku 1 przedstawiono wspólny mechanizm walidacji tokenu JWT, realizowany przed wykonaniem logiki biznesowej endpointów chronionych.



Rysunek 1: Diagram sekwencji walidacji tokenu JWT

3 Logowanie

Proces logowania umożliwia uwierzytelnienie użytkownika w systemie oraz uzyskanie tokenu JWT (*access token*), który jest następnie wykorzystywany do autoryzacji żądań kierowanych do endpointów chronionych interfejsu API.

3.1 Opis endpointu

Logowanie realizowane jest za pomocą dedykowanego endpointu autoryzacyjnego:

- Metoda HTTP: POST
- Adres URL: /api/v1/auth/login
- Typ treści: application/json

3.2 Przykładowe żądanie

Poniżej przedstawiono przykładowe żądanie HTTP służące do uwierzytelnienia użytkownika w systemie.

Listing 1: Przykładowe żądanie logowania

```
POST /api/v1/auth/login
Content-Type: application/json
Accept: application/json
```

Listing 2: Ciało żądania logowania

```
1 {
2   "login": "jan.kowalski@example.com",
3   "password": "P@ssw0rd!"
4 }
```

3.3 Przykładowa odpowiedź — sukces

W przypadku poprawnego uwierzytelnienia użytkownika system zwraca token JWT, który powinien zostać zapisany po stronie klienta i dołączany do kolejnych żądań.

Listing 3: Odpowiedź HTTP — sukces

```
200 OK
Content-Type: application/json
```

Listing 4: Ciało odpowiedzi — sukces

```
1 {
2   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
3   "type": "Bearer",
4   "expiresIn": 3600,
5   "roles": ["LANDLORD"]
6 }
```

3.4 Obsługa błędów

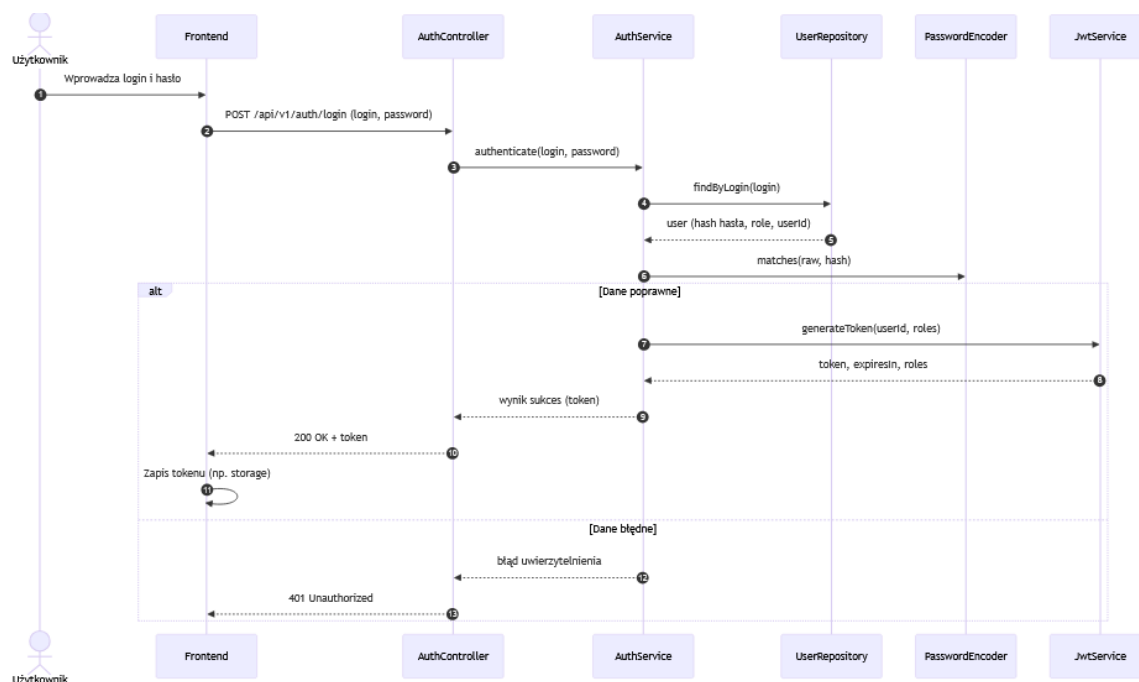
W przypadku podania nieprawidłowych danych uwierzytelniających (błędny login lub hasło) żądanie zostaje odrzucone, a system zwraca odpowiedź:

Listing 5: Odpowiedź HTTP — błędne dane logowania

```
401 Unauthorized
```

3.5 Diagram sekwencji procesu logowania

Na rysunku 2 przedstawiono diagram sekwencji obrazujący proces logowania użytkownika, obejmujący weryfikację danych uwierzytelniających oraz generowanie tokenu JWT.



Rysunek 2: Diagram sekwencji procesu logowania

3.6 Uwagi projektowe

Mechanizm logowania został zaprojektowany w sposób umożliwiający jednoznaczne oddzielenie procesu uwierzytelnienia od logiki biznesowej systemu. Token JWT zwracany w odpowiedzi zawiera informacje niezbędne do identyfikacji użytkownika oraz jego ról, co pozwala na efektywne zarządzanie dostępem do zasobów systemu.

4 Wystawienie ogłoszenia

Niniejszy rozdział opisuje proces wystawienia ogłoszenia przez użytkownika pełniącego rolę wynajmującego. Opis dotyczy wyłącznie logiki biznesowej realizowanej po stronie systemu. Mechanizm technicznej autoryzacji JWT został opisany w osobnym rozdziale i nie jest tutaj powielany.

Zakłada się, że żądanie dotarło do warstwy kontrolera z poprawnie ustawionym kontekstem użytkownika (identyfikator użytkownika oraz role).

4.1 Opis endpointu

- Metoda HTTP: POST

- Adres URL: `/api/v1/listings`
- Endpoint chroniony: tak (JWT)

4.2 Przykładowe żądanie

Listing 6: Żądanie HTTP — wystawienie ogłoszenia

```
POST /api/v1/listings
Authorization: Bearer <JWT>
Content-Type: application/json
Accept: application/json
Idempotency-Key: 8f6c4a61-4dd8-4c9a-9a9c-2d7f3a5e2b1c
```

Listing 7: Ciało żądania — wystawienie ogłoszenia

```
1 {
2   "apartmentId": "550e8400-e29b-41d4-a716-446655440000",
3   "title": "Pokój przy metrze",
4   "description": "Opis ogłoszenia...",
5   "price": 1200.00,
6   "currency": "PLN",
7   "availableFrom": "2024-10-01"
8 }
```

4.3 Przykładowa odpowiedź — sukces

W przypadku poprawnej walidacji danych oraz spełnienia warunków biznesowych ogłoszenie zostaje zapisane w systemie.

Listing 8: Odpowiedź HTTP — sukces

```
201 Created
Location: /api/v1/listings/c92f1c80-2d3a-4e5b-9a1f-8b2c3d4e5f6a
Content-Type: application/json
```

Listing 9: Ciało odpowiedzi — sukces

```
1 {
2   "listingId": "c92f1c80-2d3a-4e5b-9a1f-8b2c3d4e5f6a",
3   "status": "ACTIVE",
4   "createdAt": "2024-09-15T10:30:00Z"
5 }
```

4.4 Obsługa błędów

W trakcie przetwarzania żądania mogą wystąpić następujące błędy:

- 400 Bad Request – niepoprawne dane wejściowe (błędy walidacji),
- 403 Forbidden – brak uprawnień do wskazanego zasobu,
- 404 Not Found – wskazane mieszkanie nie istnieje.

Listing 10: Przykładowa odpowiedź — błąd walidacji

```
1 {  
2   "timestamp": "2024-09-15T10:30:00Z",  
3   "status": 400,  
4   "error": "ValidationError",  
5   "fieldErrors": [  
6     { "field": "price", "message": "Cena musi być większa od 0"  
7     },  
8     { "field": "title", "message": "Tytuł jest wymagany" }  
9   ]  
}
```

4.5 Diagram sekwencji wystawienia ogłoszenia

Na rysunku 3 przedstawiono diagram sekwencji ilustrujący przebieg procesu wystawienia ogłoszenia po stronie systemu.

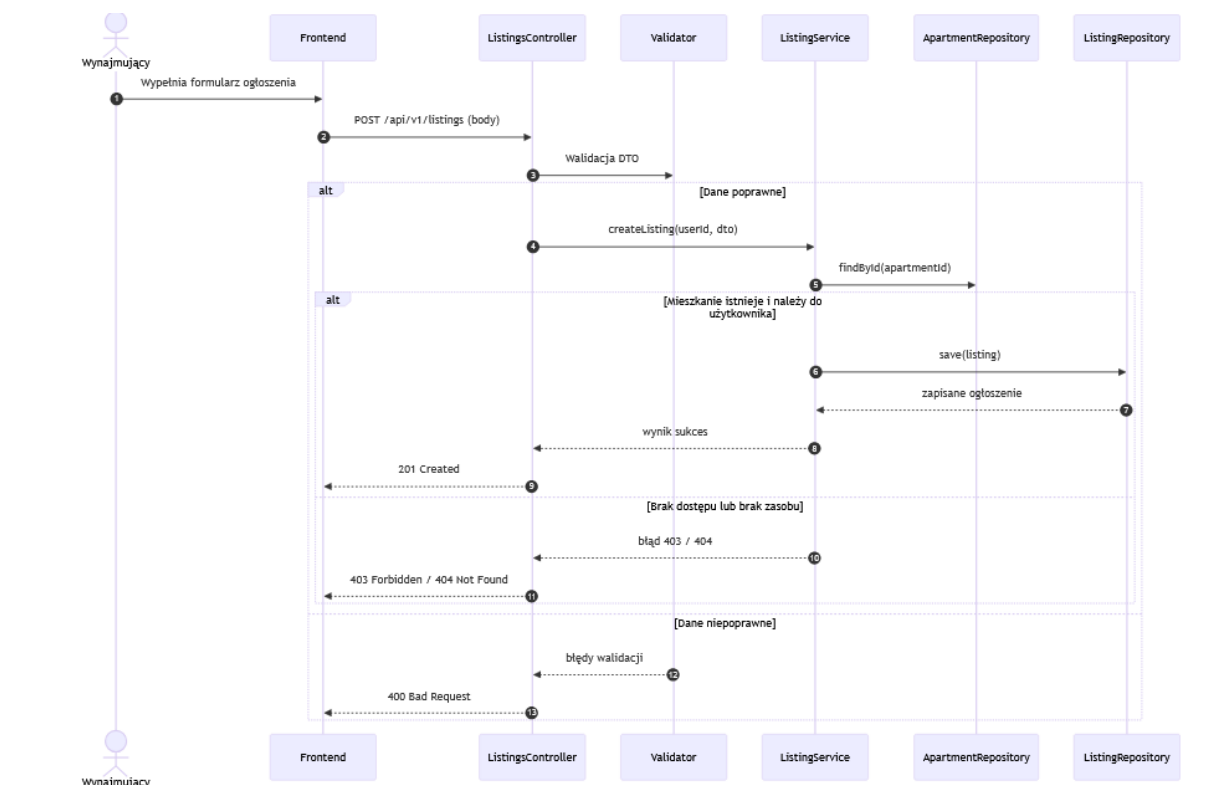
4.6 Uwagi projektowe

Zastosowanie klucza idempotencji umożliwia bezpieczne ponowienie żądania bez ryzyka utworzenia duplikatów ogłoszeń. Walidacja danych wejściowych realizowana jest przed wykonaniem logiki biznesowej, co pozwala na szybkie odrzucenie niepoprawnych żądań.

5 Zarządzanie preferencjami i dopasowywanie

Niniejszy rozdział opisuje proces dopasowywania ofert do preferencji użytkownika pełniącego rolę lokatora. Opis dotyczy logiki biznesowej realizowanej po stronie systemu. Mechanizm technicznej autoryzacji JWT został opisany w osobnym rozdziale i nie jest tutaj powielany.

Zakłada się, że żądanie zostało przetworzone przez warstwę autoryzacji, a kontekst użytkownika (identyfikator użytkownika) jest dostępny w warstwie kontrolera.



Rysunek 3: Diagram sekwencji procesu wystawienia ogłoszenia

5.1 Opis endpointu

- Metoda HTTP: GET
- Adres URL: /api/v1/matches
- Endpoint chroniony: tak (JWT)

5.2 Parametry zapytania

- page – numer strony wyników (numerowanie od 0),
- size – liczba elementów na stronie.

5.3 Przykładowe żądanie

Listing 11: Żądanie HTTP — pobranie dopasowań

```

GET /api/v1/matches?page=0&size=10
Authorization: Bearer <JWT>
Accept: application/json

```

5.4 Przykładowa odpowiedź — sukces

W odpowiedzi system zwraca listę dopasowanych ogłoszeń wraz z metadanymi stronicowania.

Listing 12: Odpowiedź HTTP — sukces

```
200 OK
Content-Type: application/json
```

Listing 13: Ciało odpowiedzi — lista dopasowań

```
1 {
2   "content": [
3     {
4       "listingId": "c92f1c80-2d3a-4e5b-9a1f-8b2c3d4e5f6a",
5       "title": "Cichy pokój",
6       "price": 1400.00,
7       "currency": "PLN",
8       "matchScore": 0.95,
9       "location": {
10        "city": "Warszawa",
11        "district": "Mokotów",
12        "street": "Puławska"
13      }
14    }
15  ],
16  "page": {
17    "size": 10,
18    "number": 0,
19    "totalElements": 45,
20    "totalPages": 5
21  }
22 }
```

5.5 Obsługa sytuacji brzegowych

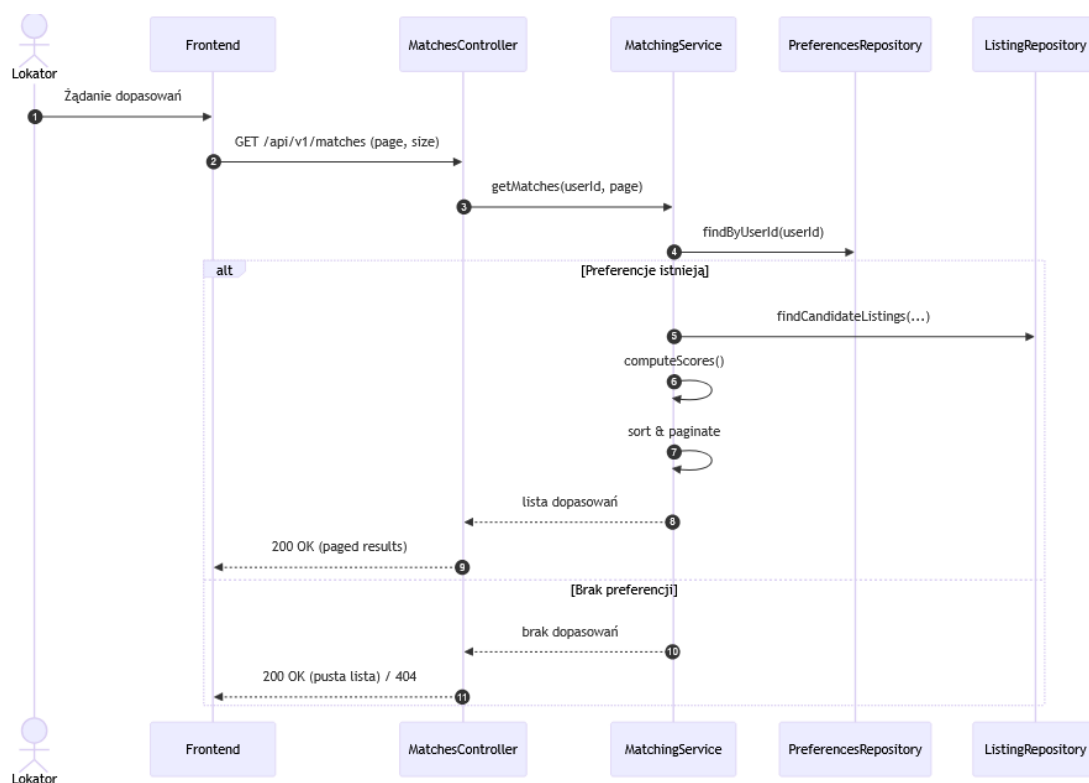
Jeżeli użytkownik nie posiada zapisanych preferencji, system może zwrócić:

- pustą listę dopasowań (kod 200 OK), lub
- odpowiedź 404 Not Found.

Wybór zachowania zależy od przyjętej polityki biznesowej systemu.

5.6 Diagram sekwencji dopasowywania

Na rysunku 4 przedstawiono diagram sekwencji ilustrujący proces pobierania dopasowanych ogłoszeń po stronie systemu.



Rysunek 4: Diagram sekwencji procesu dopasowywania

5.7 Uwagi projektowe

Proces dopasowywania opiera się na analizie preferencji użytkownika oraz ocenie stopnia zgodności ofert z wykorzystaniem algorytmu punktowego (*match score*). Wyniki są sortowane malejąco według stopnia dopasowania i zwracane w postaci stronicowanej.

6 Wynajem pokoju

Niniejszy rozdział opisuje proces wynajmu pokoju przez użytkownika pełniącego rolę lokatora. Opis dotyczy wyłącznie logiki biznesowej realizowanej po stronie systemu. Mechanizm technicznej autoryzacji JWT został opisany w osobnym rozdziale i nie jest tutaj powielany.

Zakłada się, że żądanie dotarło do warstwy kontrolera z poprawnie ustawionym kontekstem użytkownika (identyfikator użytkownika oraz role).

6.1 Opis endpointu

- Metoda HTTP: POST

- Adres URL: `/api/v1/rentals`
- Endpoint chroniony: tak (JWT)

6.2 Przykładowe żądanie

Listing 14: Żądanie HTTP — wynajem pokoju

```
POST /api/v1/rentals
Authorization: Bearer <JWT>
Content-Type: application/json
Accept: application/json
```

Listing 15: Ciało żądania — wynajem pokoju

```
1 {
2   "roomId": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
3   "startDate": "2024-11-01",
4   "endDate": "2025-06-30"
5 }
```

6.3 Przykładowa odpowiedź — sukces

W przypadku gdy wskazany pokój jest dostępny w zadanym okresie oraz spełnione są warunki biznesowe, system tworzy nową rezerwację w statusie oczekującym na zatwierdzenie.

Listing 16: Odpowiedź HTTP — sukces

```
201 Created
Location: /api/v1/rentals/aa11bb22-cc33-dd44-ee55-ff6677889900
Content-Type: application/json
```

Listing 17: Ciało odpowiedzi — sukces

```
1 {
2   "rentalId": "aa11bb22-cc33-dd44-ee55-ff6677889900",
3   "status": "PENDING_APPROVAL",
4   "createdAt": "2024-10-15T14:00:00Z",
5   "totalPrice": 9600.00,
6   "currency": "PLN",
7   "resourceLink": "/api/v1/rentals/aa11bb22-cc33-dd44-ee55-ff6677889900"
8 }
```

6.4 Obsługa błędów

W trakcie realizacji procesu wynajmu mogą wystąpić następujące błędy:

- 400 Bad Request – niepoprawny zakres dat,
- 403 Forbidden – próba wynajmu własnego pokoju,
- 404 Not Found – wskazany pokój nie istnieje,
- 409 Conflict – pokój jest niedostępny w wybranym terminie.

Listing 18: Przykładowa odpowiedź — pokój zajęty

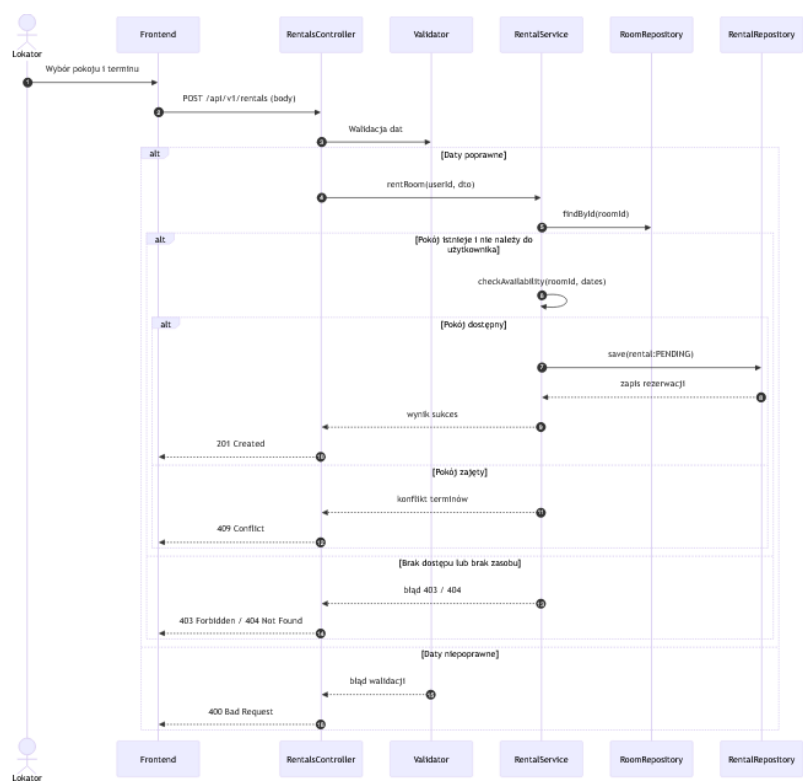
```
1 {  
2   "error": "RoomOccupied",  
3   "message": "Pokój jest niedostępny w wybranym terminie.",  
4   "unavailableDates": [  
5     { "from": "2024-12-01", "to": "2025-01-01" }  
6   ]  
7 }
```

6.5 Diagram sekwencji wynajmu pokoju

Na rysunku 5 przedstawiono diagram sekwencji ilustrujący przebieg procesu wynajmu pokoju po stronie systemu.

6.6 Uwagi projektowe

Proces wynajmu uwzględnia walidację zakresu dat, weryfikację dostępności pokoju oraz kontrolę reguł biznesowych (m.in. zakaz wynajmu własnych zasobów). Zastosowanie statusu pośredniego (PENDING_APPROVAL) umożliwia dalszą obsługę procesu, np. akceptację przez wynajmującego.



Rysunek 5: Diagram sekwencji procesu wynajmu pokoju